

Layer Importance Estimation with Imprinting for Neural Network Quantization

Hongyang Liu

University of Alberta

hliu11@ualberta.ca

Sara Elkerdawy

University of Alberta

elkerdaw@ualberta.ca

Nilanjan Ray

University of Alberta

nray1@ualberta.ca

Mostafa Elhoushi

Huawei

m.elhoushi@ieee.org

Abstract

Neural network quantization has achieved a high compression rate using fixed low bit-width representation of weights and activations while maintaining the accuracy of the high-precision original network. However, mixed precision (per-layer bit-width precision) quantization requires careful tuning to maintain accuracy while achieving further compression and higher granularity than fixed-precision quantization. We propose an accuracy-aware criterion to quantify the layer's importance rank. Our method applies imprinting per layer which acts as a proxy module for accuracy estimation in an efficient way. We rank the layers based on the accuracy gain from previous modules and iteratively quantize first those with less accuracy gain. Previous mixed-precision methods either rely on expensive search techniques such as reinforcement learning (RL) or end-to-end optimization with a lack of interpretation to the quantization configuration scheme. Our method is a one-shot, efficient, accuracy-aware information estimation and thus draws better interpretability to the selected bit-width configuration.

1. Introduction

Deep neural networks (DNN) models achieved state-of-the-art (SOTA) accuracy in numerous applications. However, DNNs commonly demand large computational costs and memory consumption. Numerous light-weight footprint architectures are designed to tackle the computation bottleneck deployment on embedded and edge devices [13, 15, 14, 32]. Other ways to address this challenge are model pruning [11, 22, 7], low-rank factorization [36, 4, 24], and quantization [19, 35, 23] that have been extensively studied. Related to our work, quantization converts weights and optionally activations from 32-bit floating points to a low precision representation. The quan-

tized model requires less memory storage and computation cost resulting in faster inference. Different layers might require different low-bit representations based on a multitude of factors such as weight distribution, information gain, and sensitivity to quantization.

Quantization stores parameters of a model with a low-bit precision and utilizes the hardware's ability to efficiently process computational operations such as convolution with low-bit operations to speed up the inference, which enables us to run neural networks on edge devices. Conventional quantization methods referred to as fixed-precision quantization in the paper such as [23, 3, 26] ignore the fact that different layers of a network have different levels of redundancy. This observation led to the development of mixed-precision quantization methods [35, 39, 5, 27, 34]. However, the challenge is to determine the bit length per layer and explore the vast design space with minimal expert knowledge. Current mixed-precision methods adopt computationally expensive optimization to search for the configuration with optimal bit-length per layer. Our method is a one-shot search optimization that is followed by a constant fine-tuning step. Computation in other quantization methods grows linear in the compression target budget. That stems from the iterative and progressive joint quantization-finetuning usually adopted in current methods. Figure 1 shows different budget-architecture pairs. Our method achieves comparable accuracy with smaller model sizes while adopting an efficient one-shot search optimization.

In this paper, we propose an efficient one-shot mixed-precision quantization method. We introduce an accuracy-aware criterion using imprinting which is introduced in few-shot learning literature [29] and adopted in pruning [7]. We insert a proxy classifier after each layer and estimate the accuracy gain per layer. The layer with the lowest rank is quantized and imprinting is re-applied again to update the ranks. This process is done iteratively until a model size